МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

ЗВІТ

Комп'ютерний практикум №2

«Розробка веб-сайту з базою даних. RESTful API»

з дисципліни

«Технології розробки WEB-орієнтованих систем»

Тема №69: «Спортзал»
Веб-сайт «Ритм»

Виконавець: **Суворов Дмитро**
Група: ЦІ-31
Викладач: С. В. Суліма

Київ — 2026

Зміст

1 Тема, мета та виконані завдання **2**

2 База даних **2**
2.1 Сутності, атрибути та зв'язки . . . 2
2.2 Обґрунтування третьої нормальної форми . . . 3
2.3 ER-діаграма . . . 4

3 Архітектура та запуск **4**

4 REST API **5**
4.1 Приклади запитів і відповідей . . . 5
4.2 Валідація та обробка помилок . . . 7

5 Автентифікація та авторизація **7**

6 Перевірка API та інтеграції **7**
GET: каталог із PostgreSQL . . . 8
POST: створення тренування, 201 . . . 9
PATCH: зміна тренування, 200 . . . 10
POST: створення запису, 201 . . . 11
PATCH: зміна запису, 200 . . . 12
DELETE: видалення запису, 204 . . . 13
DELETE: видалення тренування, 204 . . . 14
Валідація: некоректна ціна, 400 . . . 15
Без токена: 401 . . . 16
Недостатні права: 403 . . . 17
Відсутній ресурс: 404 . . . 18

7 Оновлений клієнтський інтерфейс **19**
Каталог із бази даних . . . 20
Деталі тренування . . . 21
Реєстрація та вхід . . . 22
Форма запису на заняття . . . 23
Кабінет із власними записами . . . 24
Керування каталогом . . . 25
Помилка мережі та повторний запит . . . 26

8 Висновки **29**

9 Посилання та матеріали **29**

1. Тема, мета та виконані завдання

Продовжено проєкт спортзалу «Ритм» з КП №1. Мета роботи — перейти від статичного сайту до клієнт-серверного застосунку з реляційною базою даних, REST API та автентифікацією.

- Спроєктовано чотири пов'язані таблиці PostgreSQL та ER-діаграму.
- Реалізовано Express-сервер, CRUD для тренувань і записів на заняття.
- Додано реєстрацію, вхід, JWT та розмежування доступу member/admin.
- Каталог, деталі, форма запису та кабінет інтегровані з API через Fetch.
- Проведено перевірку API в Insomnia та перевірку клієнта у двох браузерах.
- Підготовлено документацію API, інструкцію запуску та окремий звіт №2.

Виконавець	Суворов Дмитро
Група	ЦІ-31
Внесок	Проєктування БД, сервер, клієнтська інтеграція, перевірка і документація

Матеріали першої роботи збережено у `lab-report/`; нові — у `lab-report-2/`. Статичну версію позначено Git-тегом `lab1-v1.0`.

2. База даних

2.1. Сутності, атрибути та зв'язки

Таблиця	Основні атрибути	Ключі
users	id, name, email, password_hash, role, created_at	PK id; UNIQUE email
categories	id, name	PK id; UNIQUE name
trainings	id, category_id, name, description, duration_minutes, price_cents, image, created_at	PK id; FK category_id; UNIQUE name
bookings	id, user_id, training_id, session_date, phone, note, created_at	PK id; FK user_id, training_id; UNIQUE(user_id, training_id, session_date)

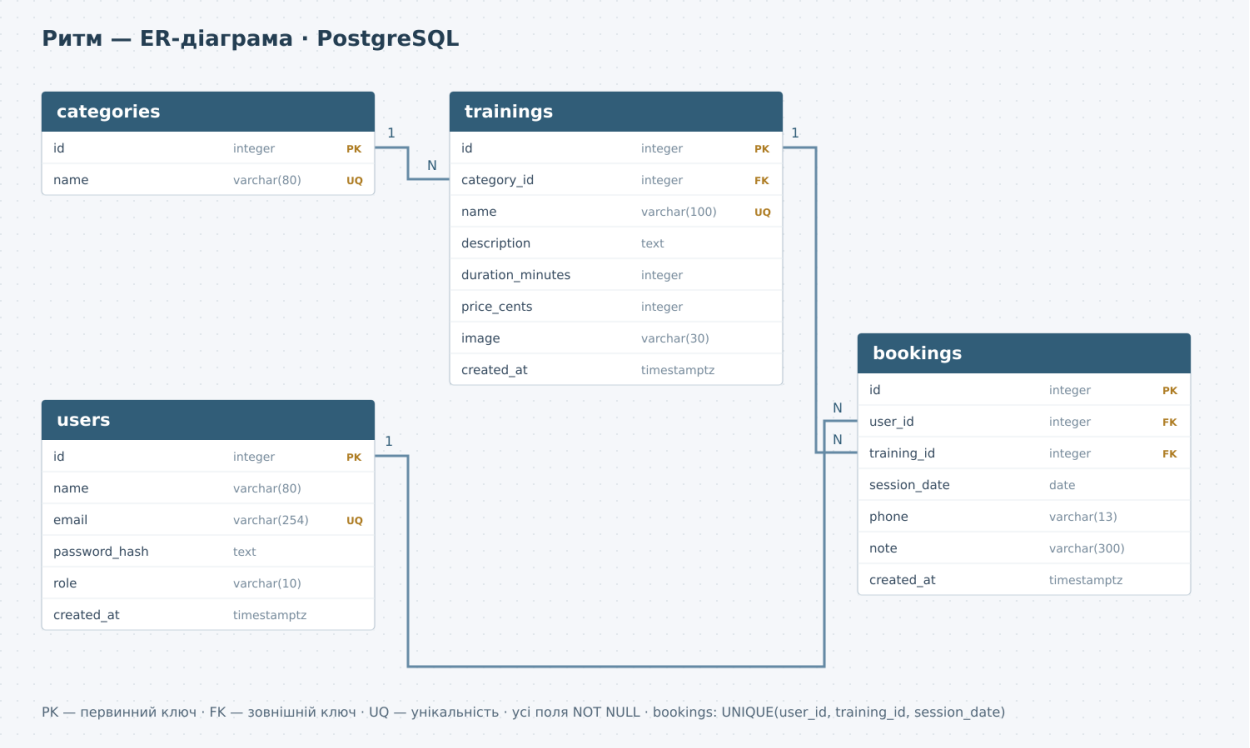
Категорія має багато тренувань; користувач має багато записів; тренування має багато записів. Усі зовнішні ключі мають `ON DELETE RESTRICT`. Неможливо видалити тренування, доки на нього існують записи. Дублювання запису на ту саму дату обмежено складеним унікальним ключем.

Ціна зберігається цілими копійками, тривалість — хвилинами, дата заняття — типом `date`, час створення — `timestamptz`. Поля обов'язкові; коментар може бути порожнім рядком. CHECK обмежує роль, тривалість, ціну, формат телефону й дозволені зображення.

2.2. Обґрунтування третьої нормальної форми

Усі атрибути атомарні (1НФ). Неключові атрибути повністю залежать від первинного ключа, часткових залежностей немає (2НФ). Назви категорій, імена користувачів та назви/ціни тренувань не дублюються у залежних таблицях: API отримує їх через JOIN. Транзитивних залежностей неключових атрибутів немає (3НФ). Телефон належить конкретному запису і може відрізнятися між записами одного користувача.

2.3. ER-діаграма



На діаграмі наведено поля, типи й зв'язки 1:N. Модель `database/schema.dbml` можна імпортувати у dbdiagram.io; виконувана схема з обмеженнями міститься у `database/schema.sql`.

3. Архітектура та запуск

Стек: Node.js 22+, Express 5, PostgreSQL 17, драйвер `pg`, Zod, bcryptjs, jsonwebtoken, Helmet. Сервер одночасно роздає дозволені HTML/CSS/JS/PNG/WOFF2 та API на одному origin. Клієнт не має прямого доступу до БД.



- `server/index.js`: маршрути, автентифікація, права доступу, валідація та обробник помилок.
- `server/db.js`: пул з'єднань і параметризовані SQL-запити.
- `scripts/setup.js`: створення локальних випадкових секретів.
- `scripts/init-db.js`: схема, початкові категорії, шість тренувань і адміністратор.
- `scripts/api.js`: Fetch, Bearer JWT, таймаут, обробка 401 і мережевих помилок.
- `scripts/app.js`: каталог, деталі, кабінет, форма та редактор адміністратора.

Порядок запуску:

```
npm ci
npm run setup
docker compose up -d --wait db
npm run db:init
npm start
```

Адреса: `http://localhost:3000`. Пароль адміністратора та секрети зберігаються тільки в локальному `.env`, виключеному з Git. Звичайний користувач реєструється у кабінеті. PostgreSQL зберігає дані у Docker volume, тому перезапуск не видаляє записи.

4. REST API

API використовує ресурсні URL, методи GET/POST/PATCH/DELETE та JSON. Тренування підтримують повний CRUD; записи також підтримують повний CRUD з перевіркою власника. Категорії доступні для читання.

Метод	URL (префікс /api)	Доступ	Результат
GET	/health	усі	200: БД доступна
POST	/auth/register	усі	201: user, token
POST	/auth/login	усі	200: user, token
GET	/auth/me	JWT	200: профіль
GET	/categories	усі	200: список
GET	/trainings	усі	200: каталог
GET	/trainings/:id	усі	200: тренування
POST	/trainings	admin	201 + Location
PATCH	/trainings/:id	admin	200: оновлення
DELETE	/trainings/:id	admin	204: без тіла
GET	/bookings	JWT	200: власні записи
GET	/bookings/:id	власник	200: запис
POST	/bookings	JWT	201 + Location
PATCH	/bookings/:id	власник	200: оновлення
DELETE	/bookings/:id	власник	204: без тіла

4.1. Приклади запитів і відповідей

Реєстрація:

```
POST /api/auth/register
{"name":"Dmytro","email":"dmytro@example.com",
 "password":"Example-Password-123"}
```

```
201 Created
{"user":{"id":2,"name":"Dmytro",
 "email":"dmytro@example.com","role":"member"},
 "token":"<JWT>","expires_in":3600}
```

Створення тренування (admin):

```
POST /api/trainings
{"category_id":1,"name":"Functional training",
 "description":"Training with an instructor.",
 "duration_minutes":50,"price_cents":35000,
 "image":"personal.png"}
201 Created
Location: /api/trainings/8
{"data":{"id":8,"category_id":1,
 "name":"Functional training",
 "description":"Training with an instructor.",
 "duration_minutes":50,"price_cents":35000,
 "image":"personal.png","created_at":"..."}}
PATCH /api/trainings/8
{"price_cents":36000}
200 OK
DELETE /api/trainings/8
204 No Content
```

Створення запису (дата повинна бути актуальною):

```
POST /api/bookings
{"training_id":1,"session_date":"2026-11-20",
 "phone":"+380671234567","note":"First session"}
201 Created
{"data":{"id":4,"user_id":2,"training_id":1,
 "session_date":"2026-11-20","phone":"+380671234567",
 "note":"First session","created_at":"...",
 "training_name":"...","price_cents":25000,
 "duration_minutes":60}}
PATCH /api/bookings/4
{"note":"Updated note"}
200 OK
DELETE /api/bookings/4
204 No Content
```

Повні приклади, параметри, коди помилок та імпортована колекція наведені у `docs/API.md` і `docs/insomnia-collection.json`.

4.2. Валідація та обробка помилок

Сервер перевіряє типи, обов'язкові поля й межі; невідомі поля та порожній PATCH відхиляються. Пароль: мінімум 12 символів, максимум 72 UTF-8 байти. Назва тренування: 2–100 символів; опис: 10–2000; тривалість: 15–240 хв; ціна: 0–1000000 копійок. Для запису перевіряються календарна дата від сьогодні до року вперед, телефон формату +380 та дев'ять цифр, коментар до 300 символів.

400 означає некоректні вхідні дані; 401 — відсутню/невдалу автентифікацію; 403 — недостатні права; 404 — відсутній або чужий ресурс; 409 — конфлікт унікальності чи зовнішнього ключа; 500 — внутрішню помилку. SQL і стек помилки не передаються клієнту. Повідомлення містить загальне пояснення та, для валідації, масив помилок полів.

5. Автентифікація та авторизація

Після реєстрації пароль хешується bcrypt з cost 12, зберігається тільки password_hash. Email нормалізується до нижнього регістру. Реєстрація не дозволяє призначити собі роль admin.

Після входу сервер видає JWT HS256 із subject, issuer, audience та строком дії одна година. Клієнт зберігає токен у sessionStorage і надсилає його в Authorization: Bearer. Сервер перевіряє підпис, строк, issuer/audience та актуального користувача. Таблиця сесій не ведеться: кожен запит самодостатній. Вихід видаляє токен з браузера; виданий токен залишається чинним до завершення строку дії.

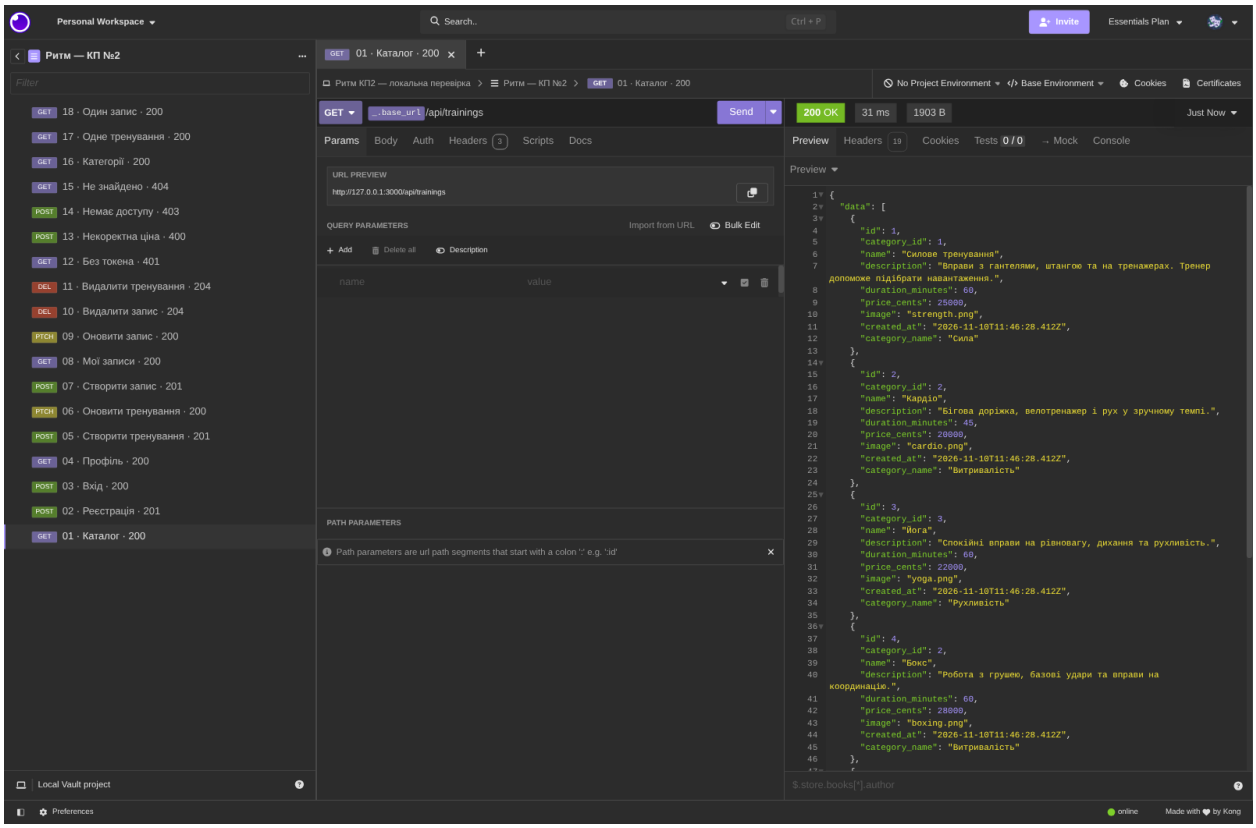
Каталог змінює тільки admin. У SQL-запитах до bookings одночасно перевіряються id ресурсу та user_id з перевіреного токена. Чужий запис повертає 404. Додаткові заходи: параметризовані SQL-запити, Helmet/CSP, обмеження тіла JSON до 16 КБ та rate limit для входу/реєстрації. Для розгортання в інтернеті потрібен HTTPS.

6. Перевірка API та інтеграції

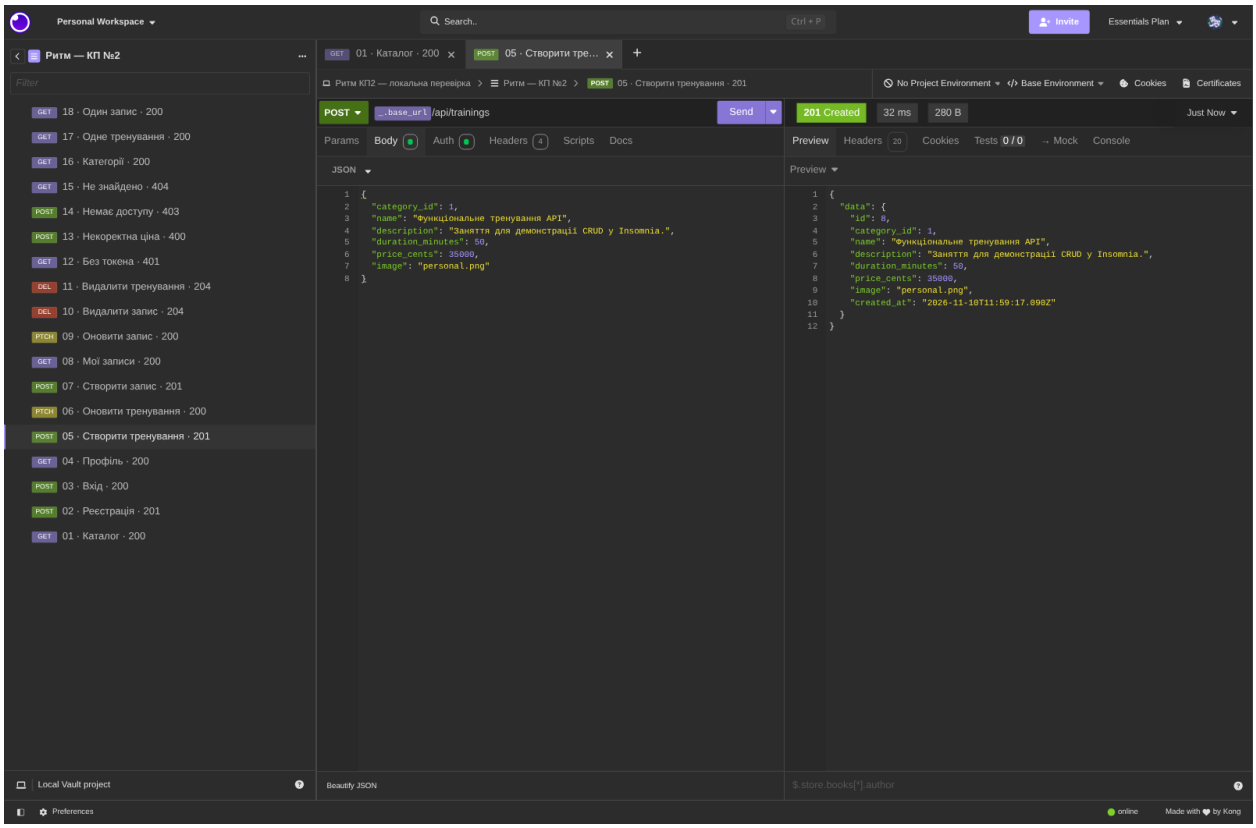
Запити виконано у Insomnia Local Vault. Перевірено GET каталогу, створення, редагування та видалення тренувань і записів та негативні відповіді 400, 401, 403 і 404. Скриншоти фактичних відповідей наведено нижче. Додаткові перевірочні HTTP-запити підтвердили 409 при дублюванні, обмеження FK та ізоляцію записів різних користувачів. Журнал 27 сценаріїв: `api-verification.json`. Це перевірка під час розроблення; окремі автоматизовані unit/API-набори КП №3 ще не реалізовувалися.

Клієнт перевірено у Chromium та Firefox при ширинах 390 і 1440 px. Підтверджено завантаження даних, зображень, вхід, керування своїми записами, адміністративний CRUD, повідомлення про помилку мережі та повторне завантаження. Скриншоти відображають дані реальної локальної PostgreSQL-бази. Початкові шість тренувань створені seed-скриптом.

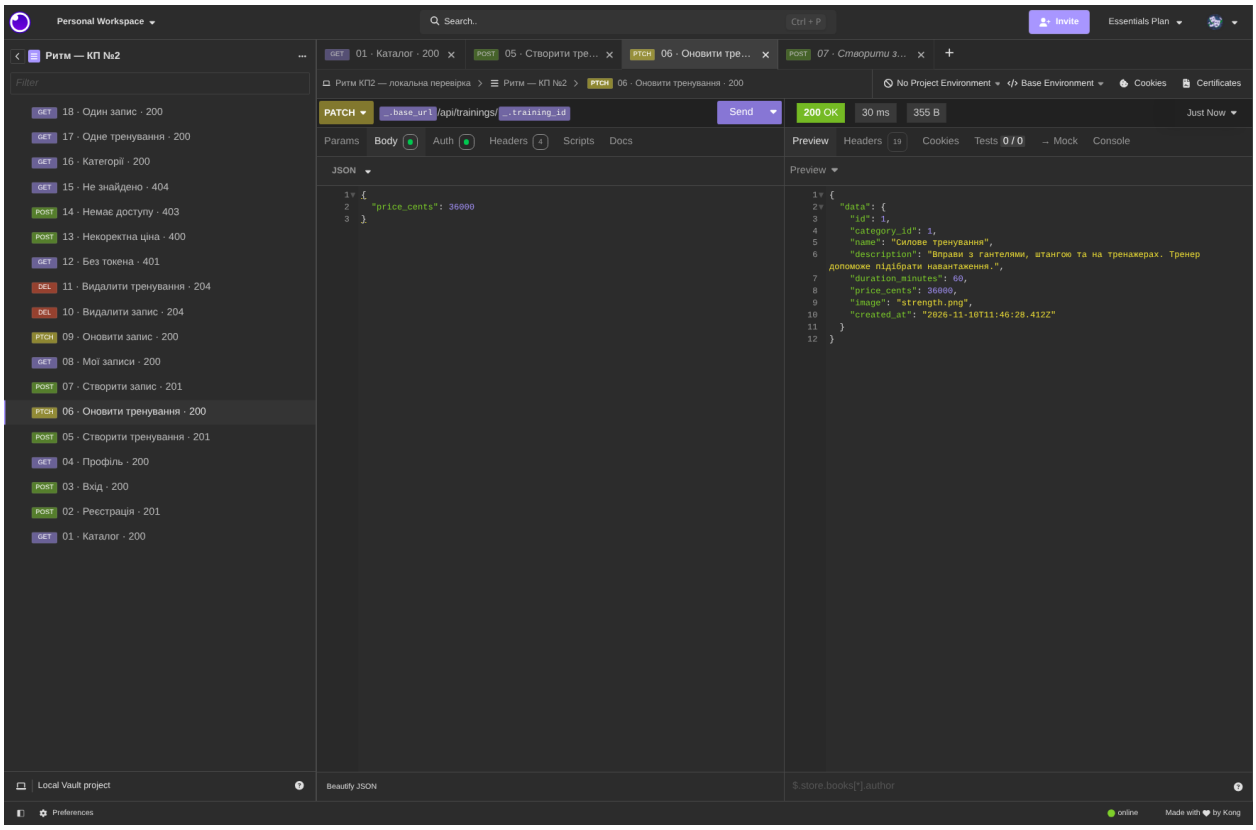
GET: каталог із PostgreSQL



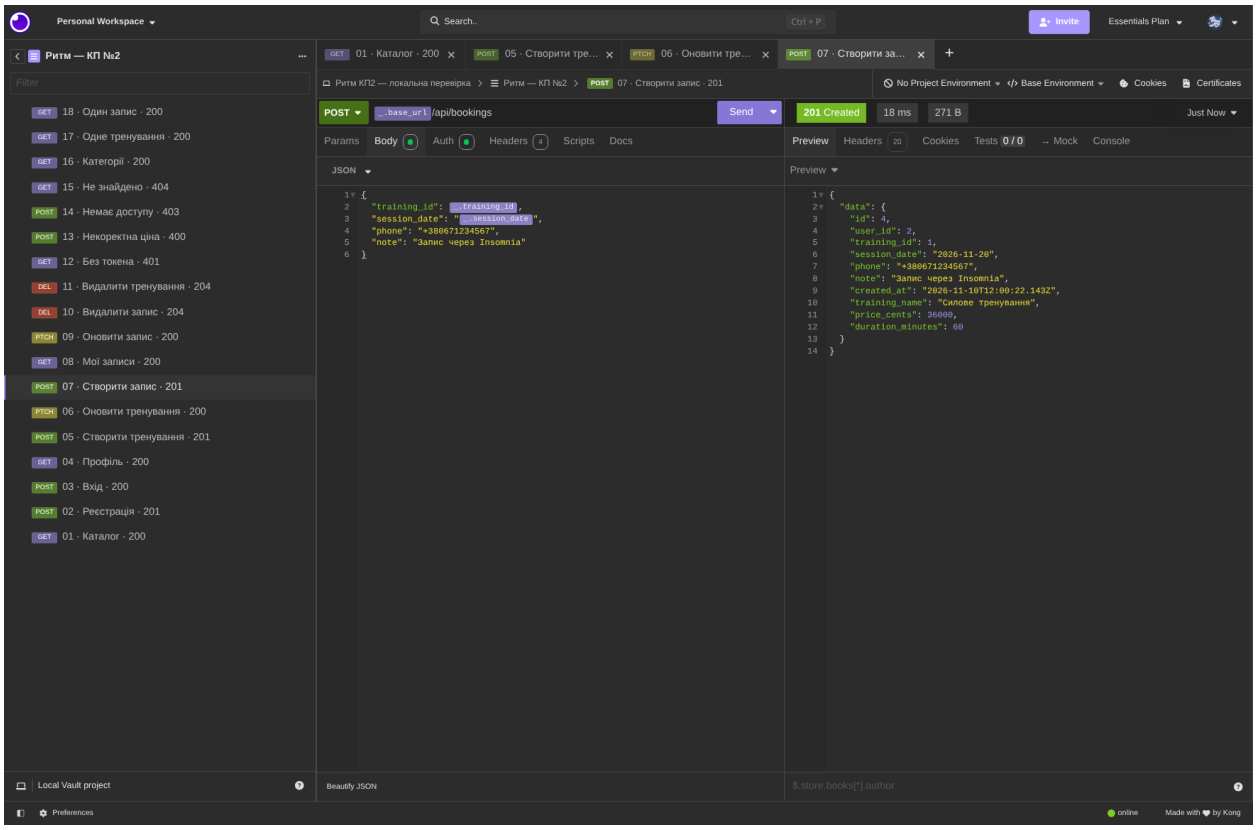
POST: створення тренування, 201



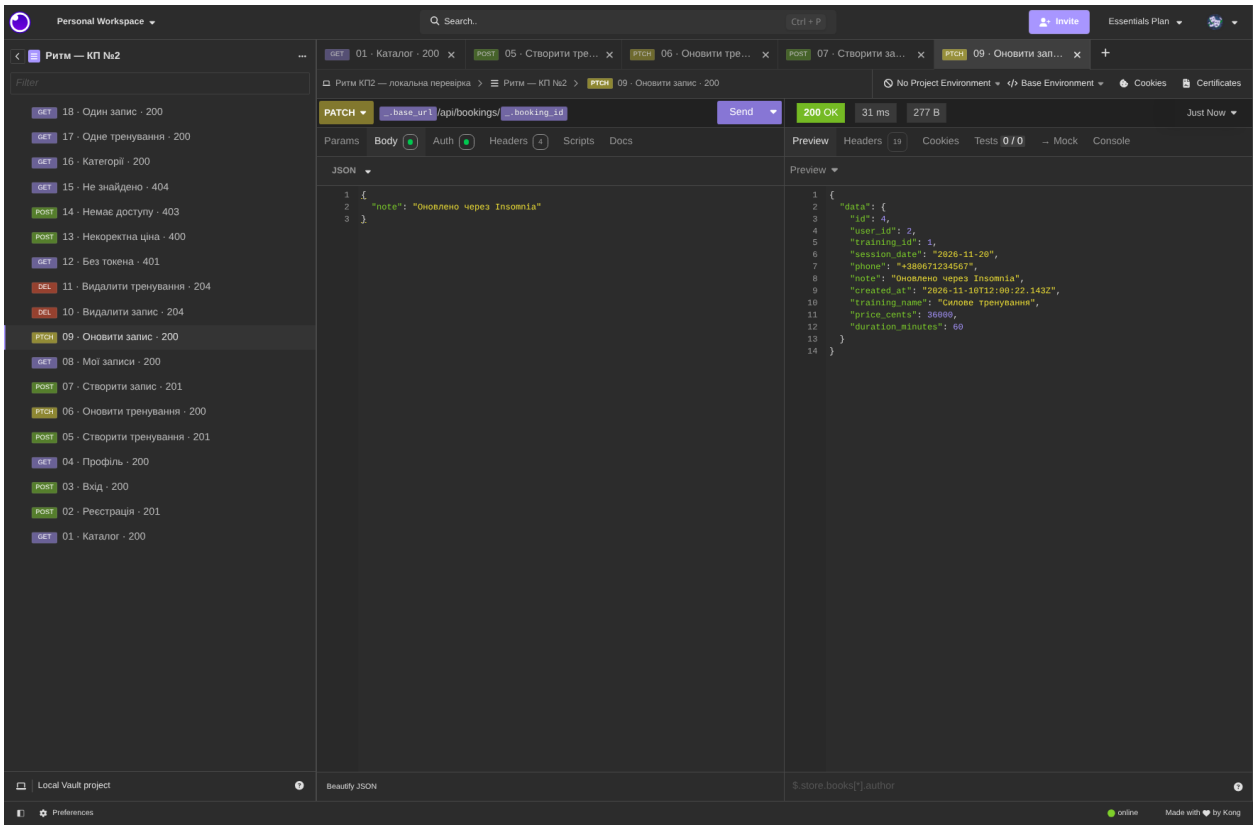
PATCH: зміна тренування, 200



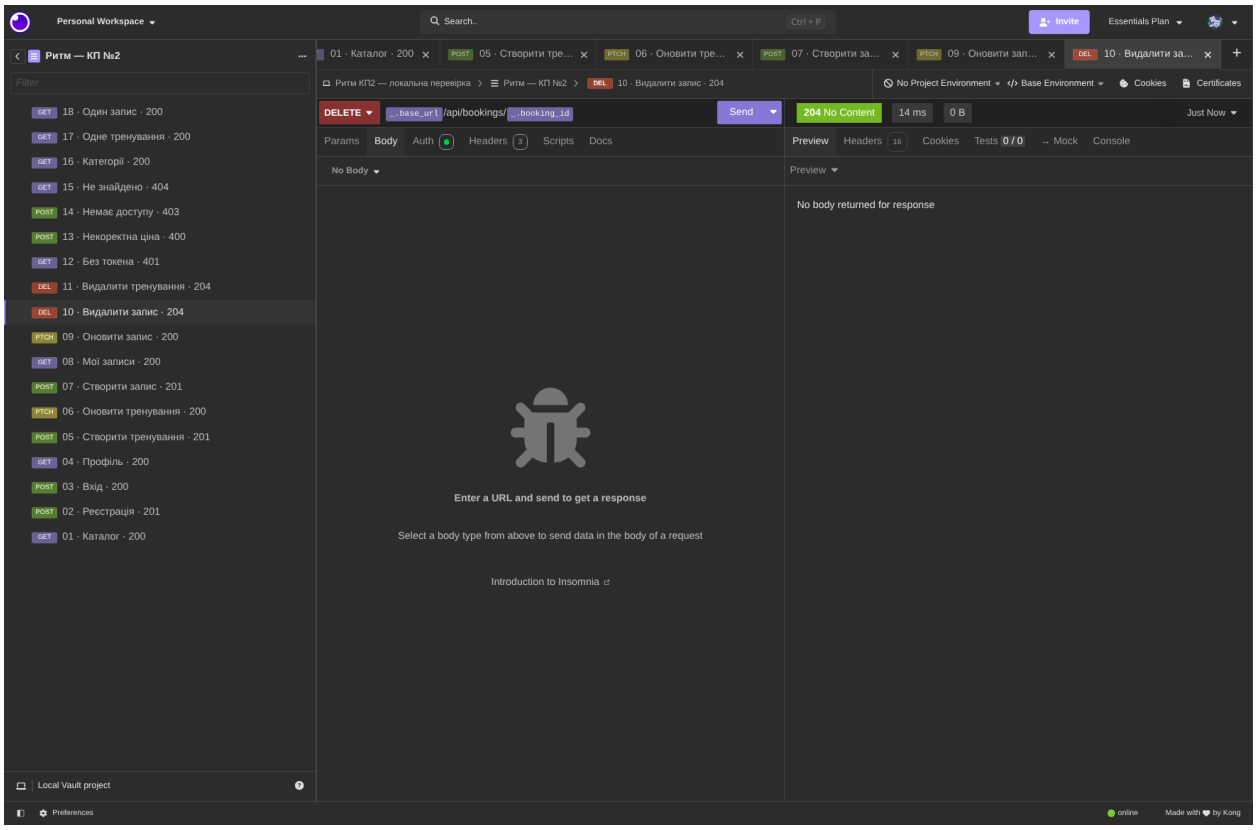
POST: створення запису, 201



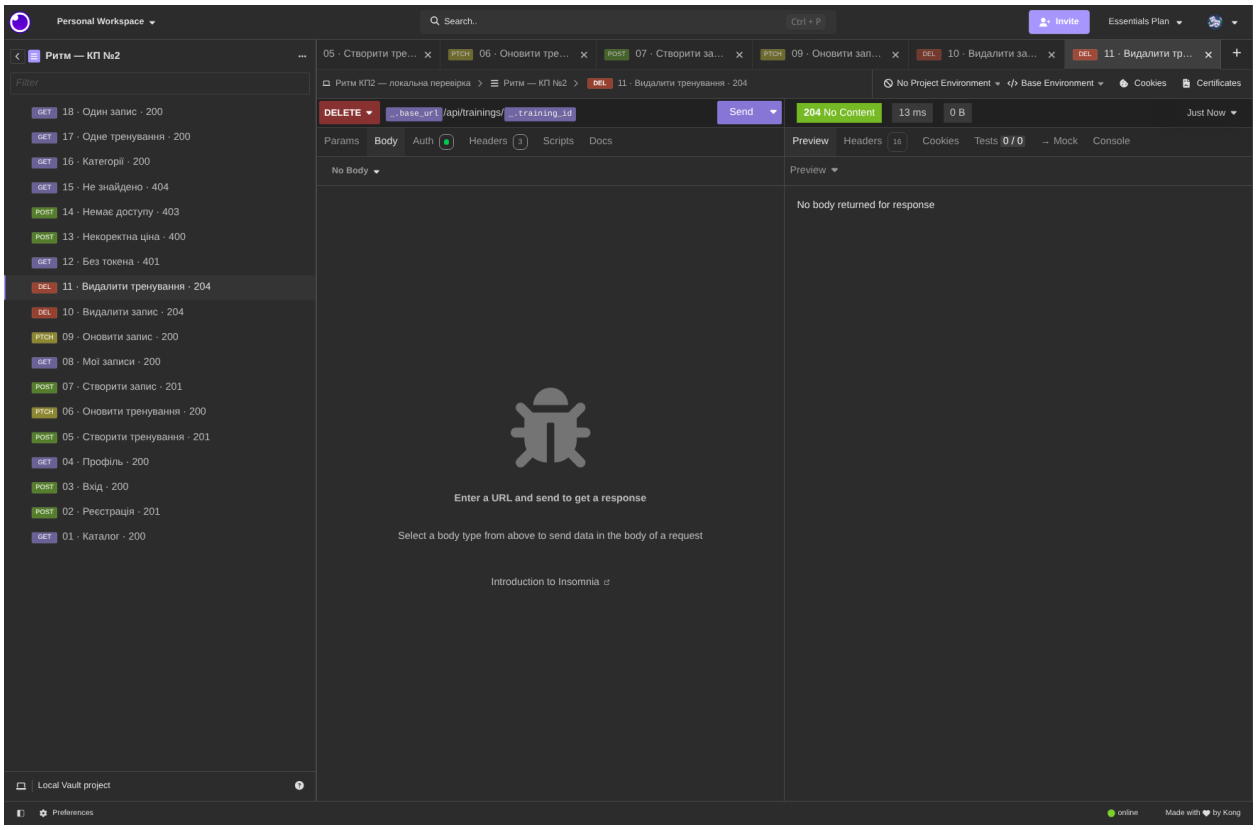
PATCH: зміна запису, 200



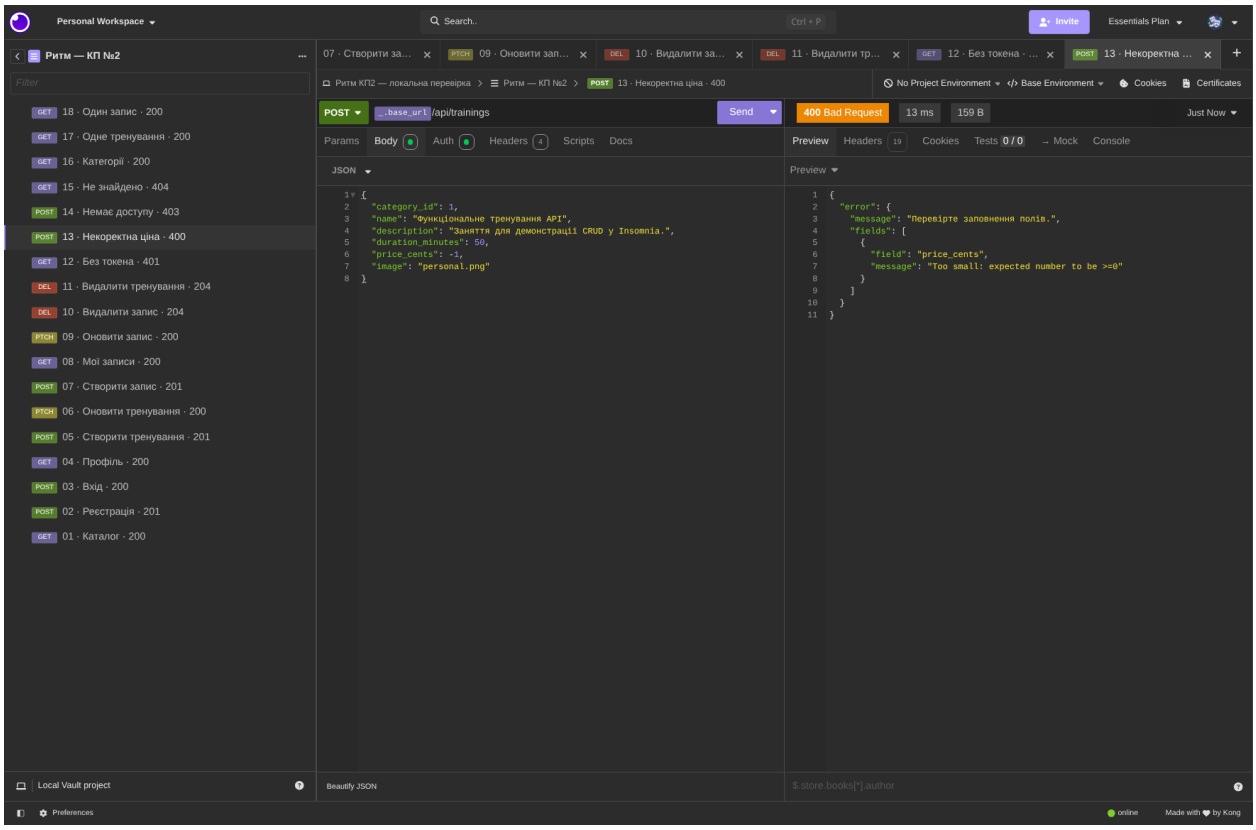
DELETE: видалення запису, 204



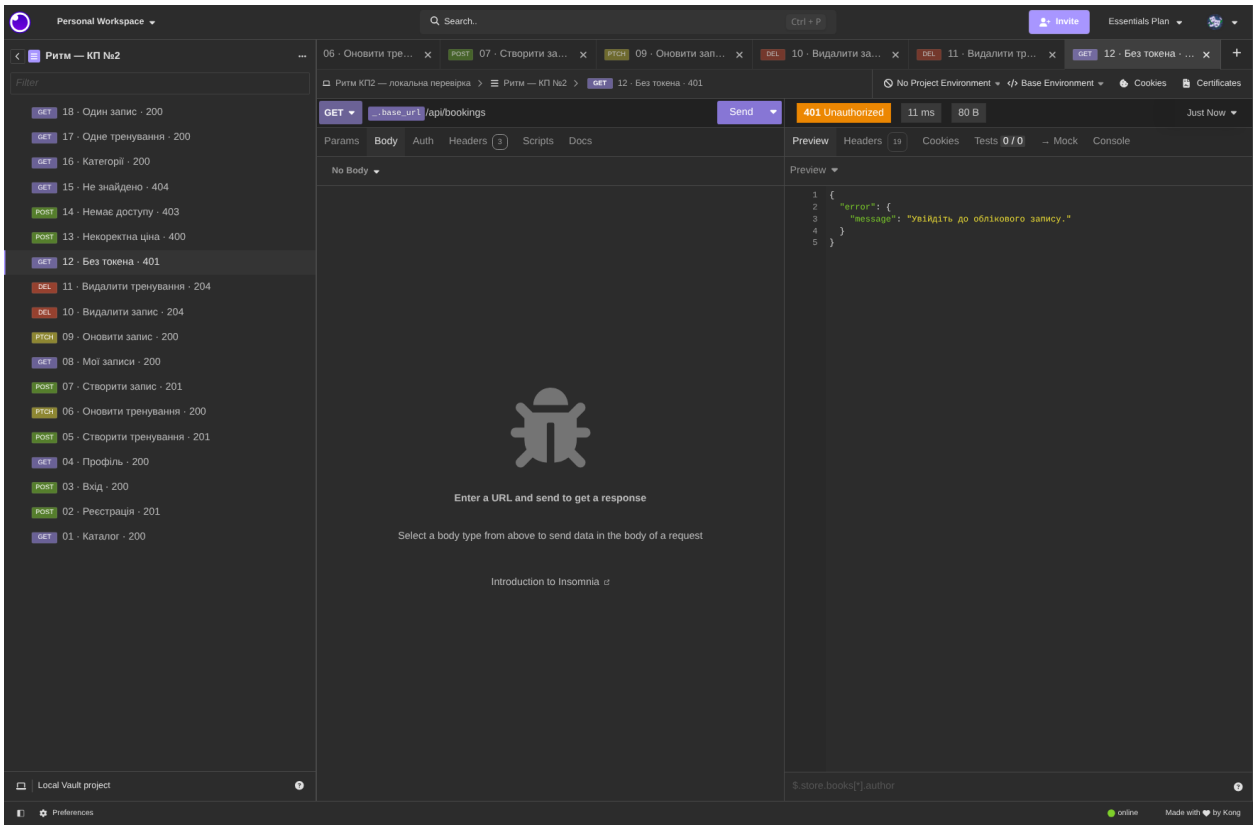
DELETE: видалення тренування, 204



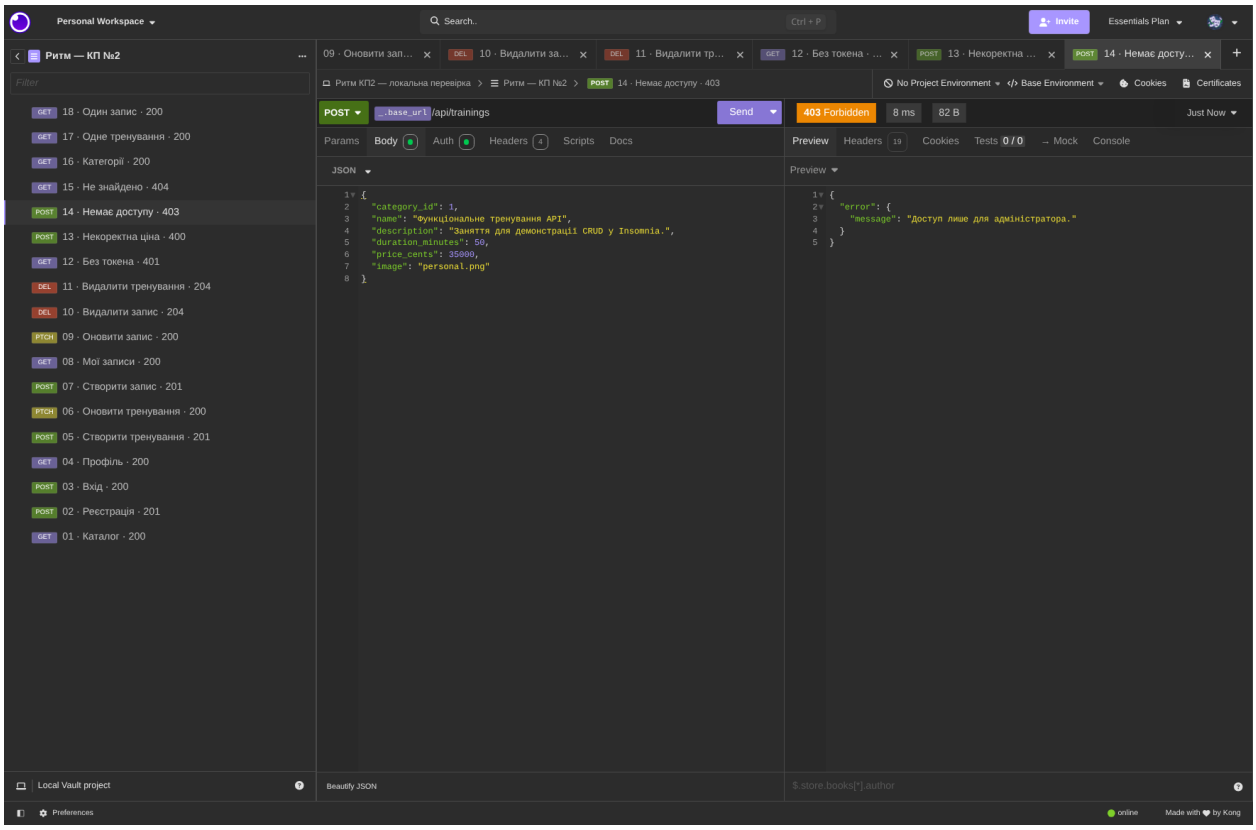
Валідація: некоректна ціна, 400



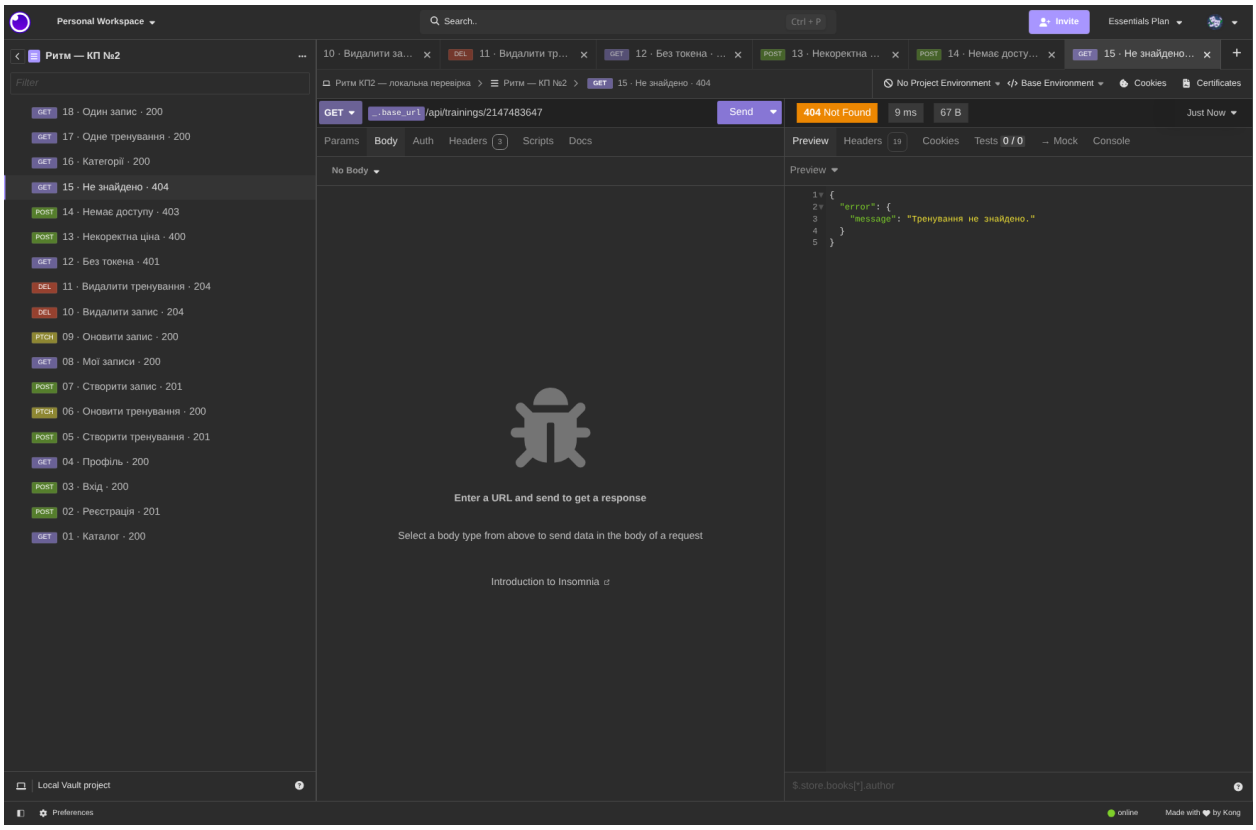
Без токена: 401



Недостатні права: 403

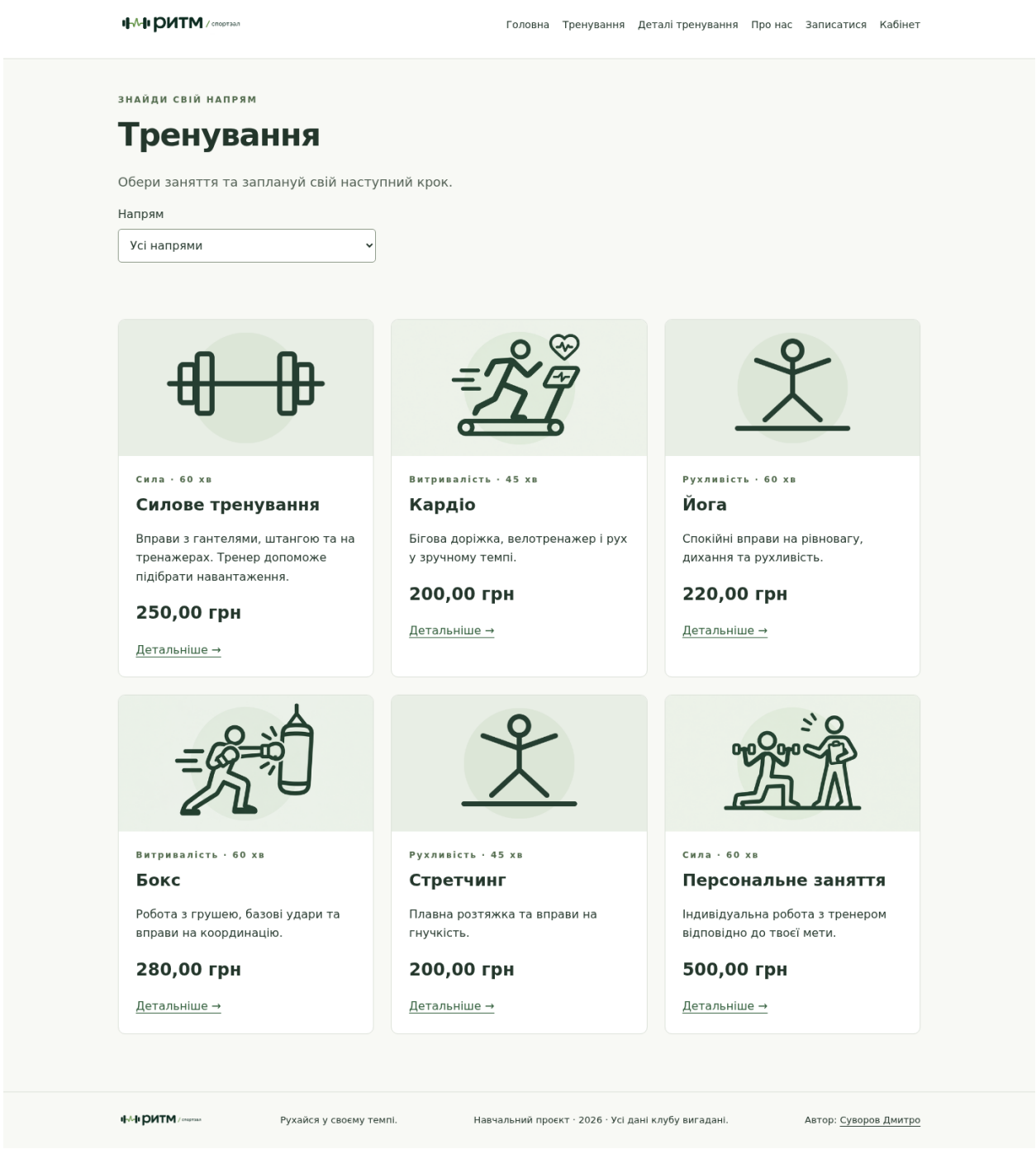


Відсутній ресурс: 404

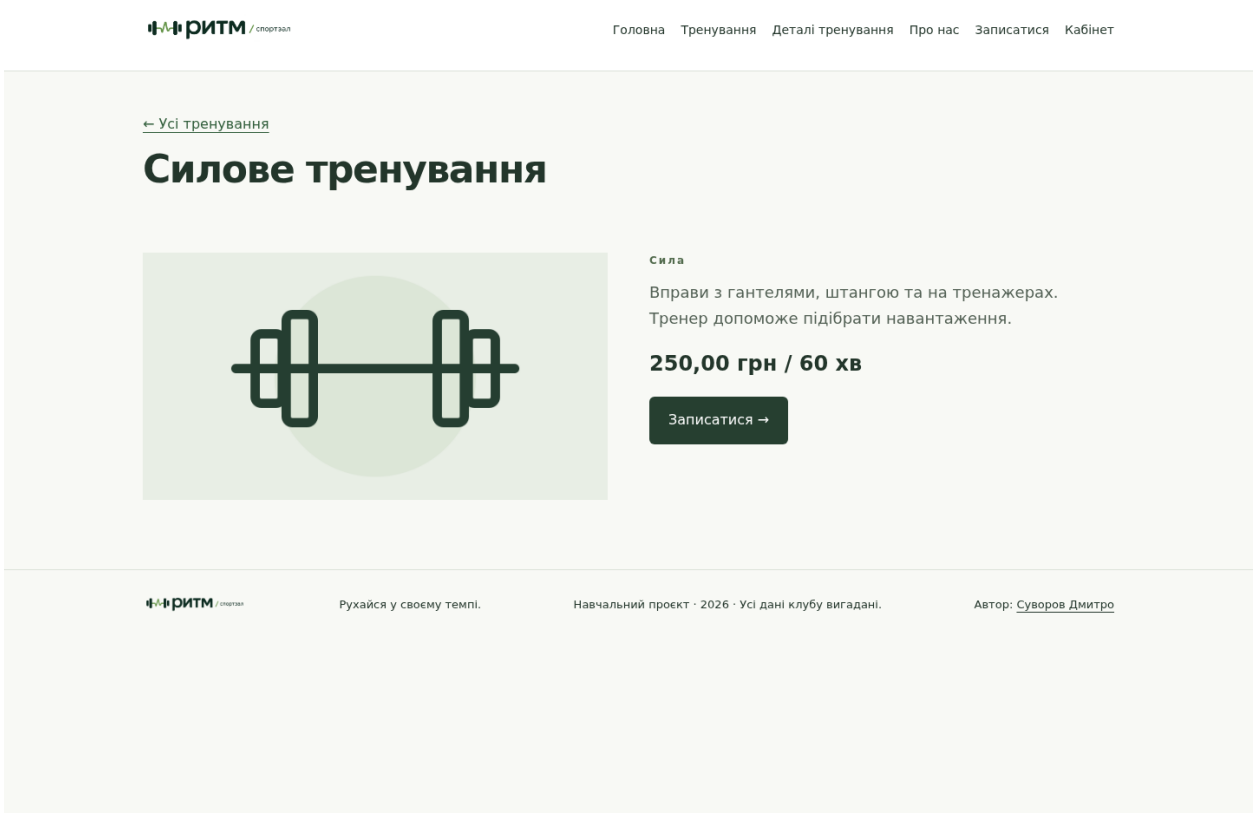


7. Оновлений клієнтський інтерфейс

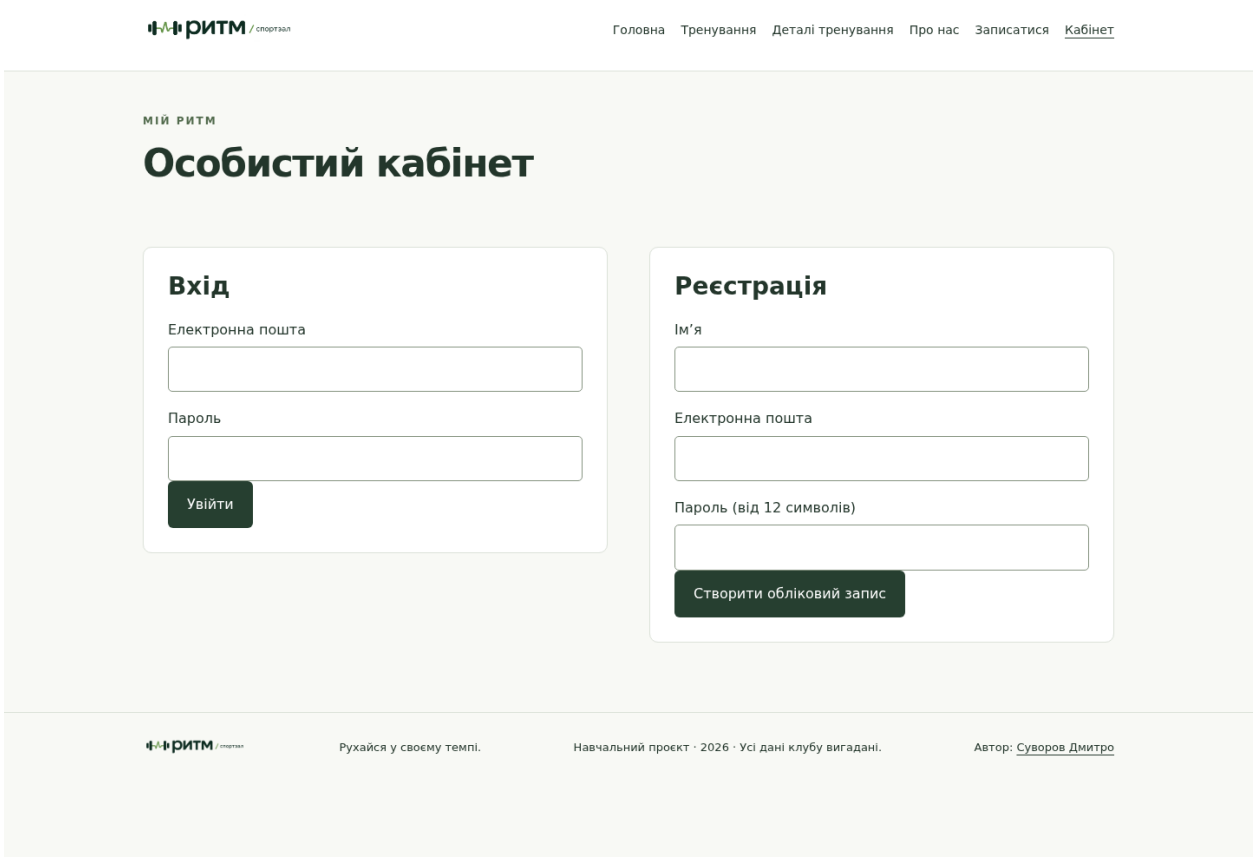
Каталог із бази даних



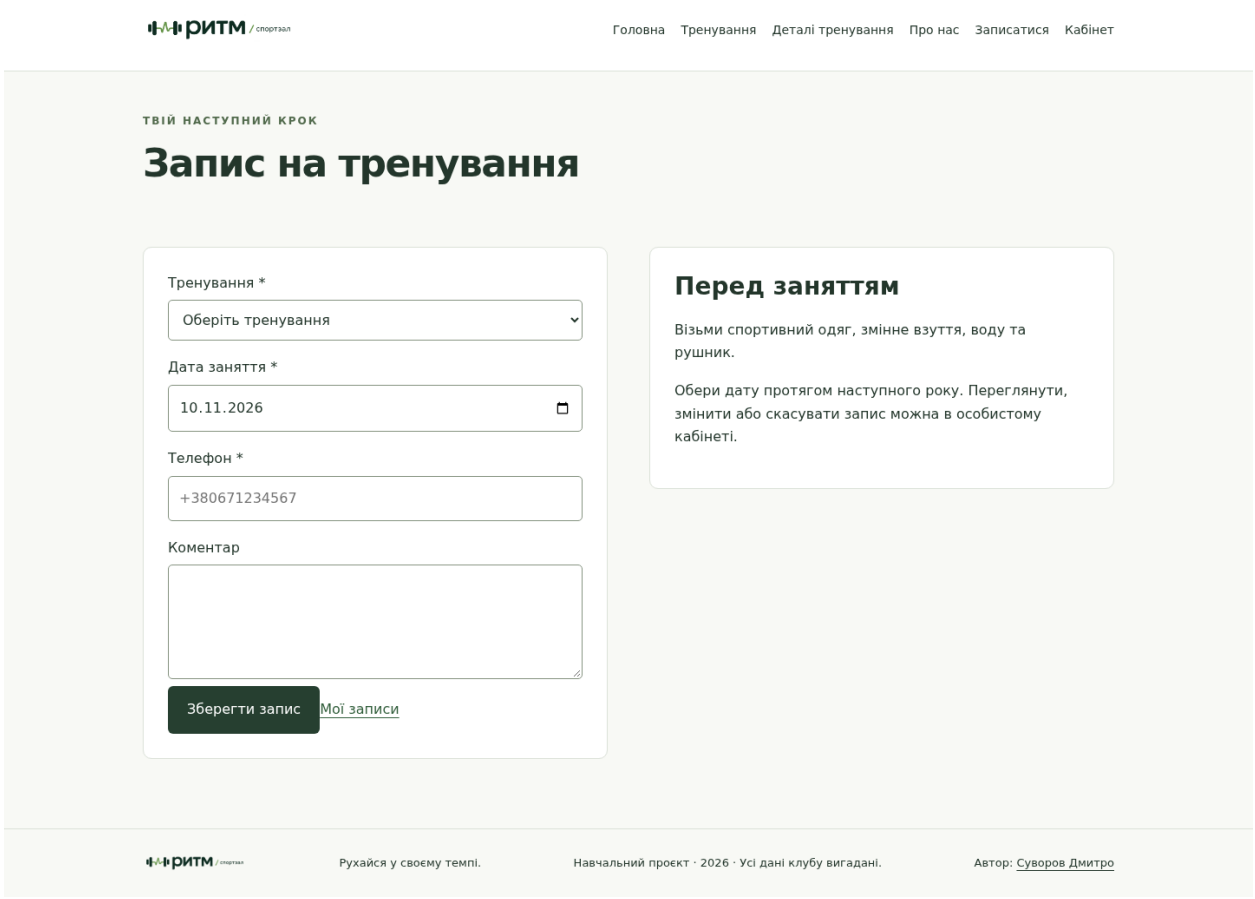
Деталі тренування



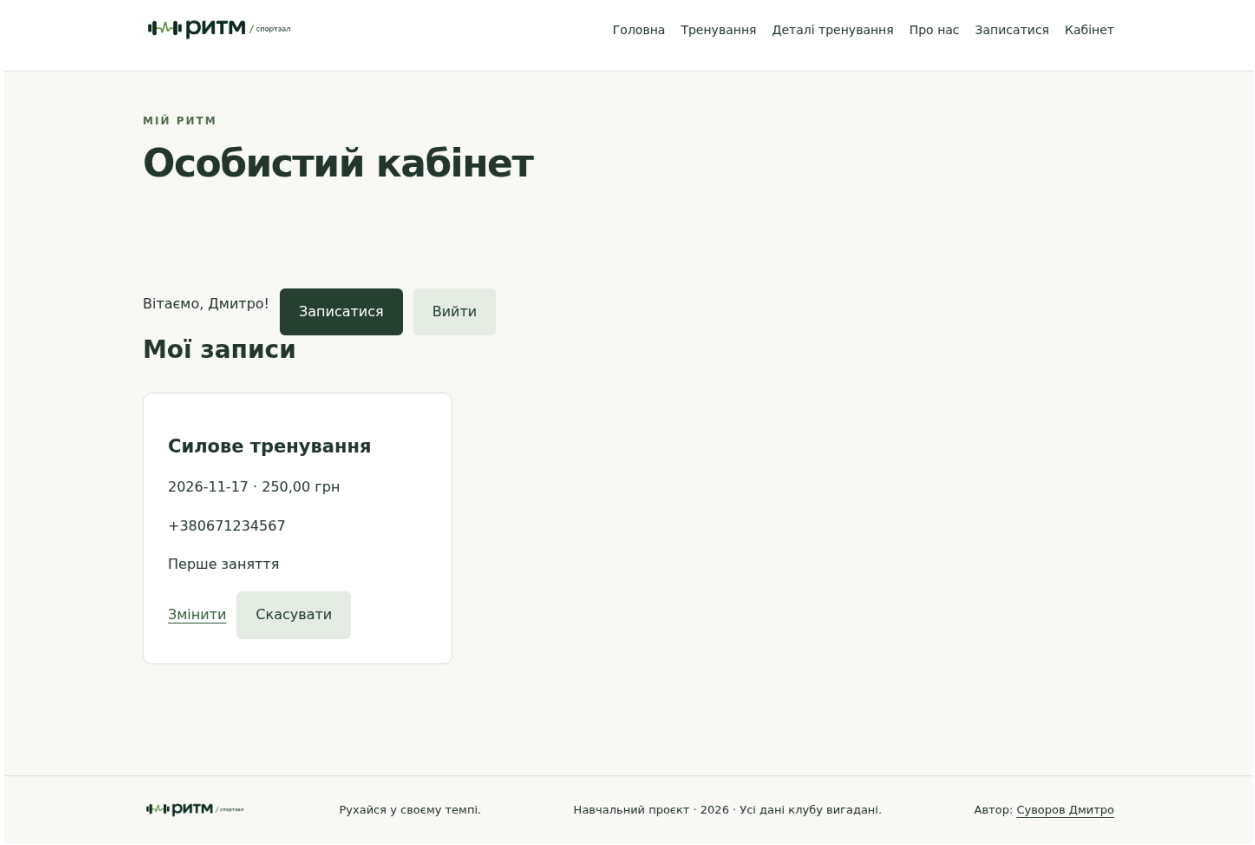
Реєстрація та вхід



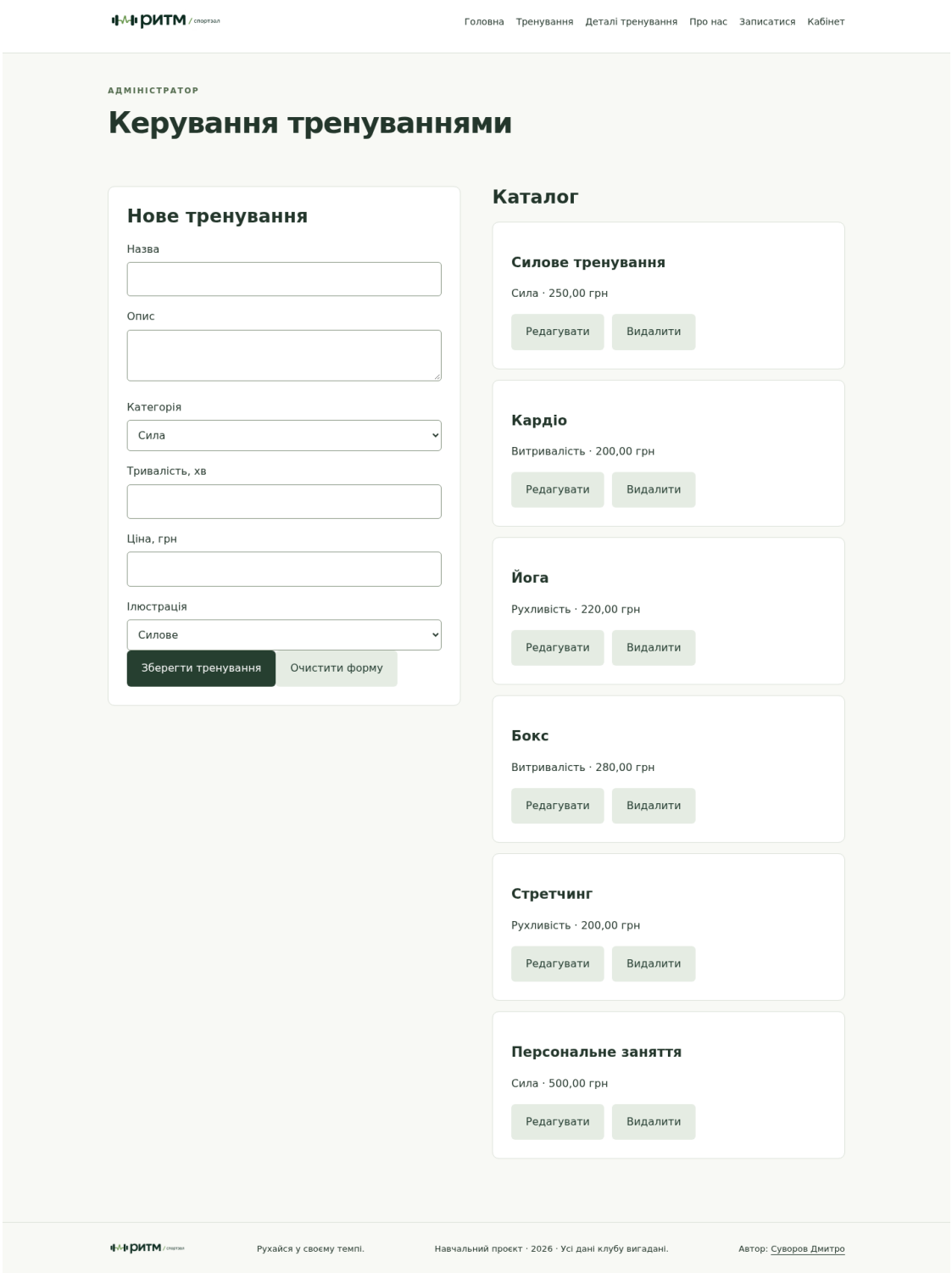
Форма запису на заняття



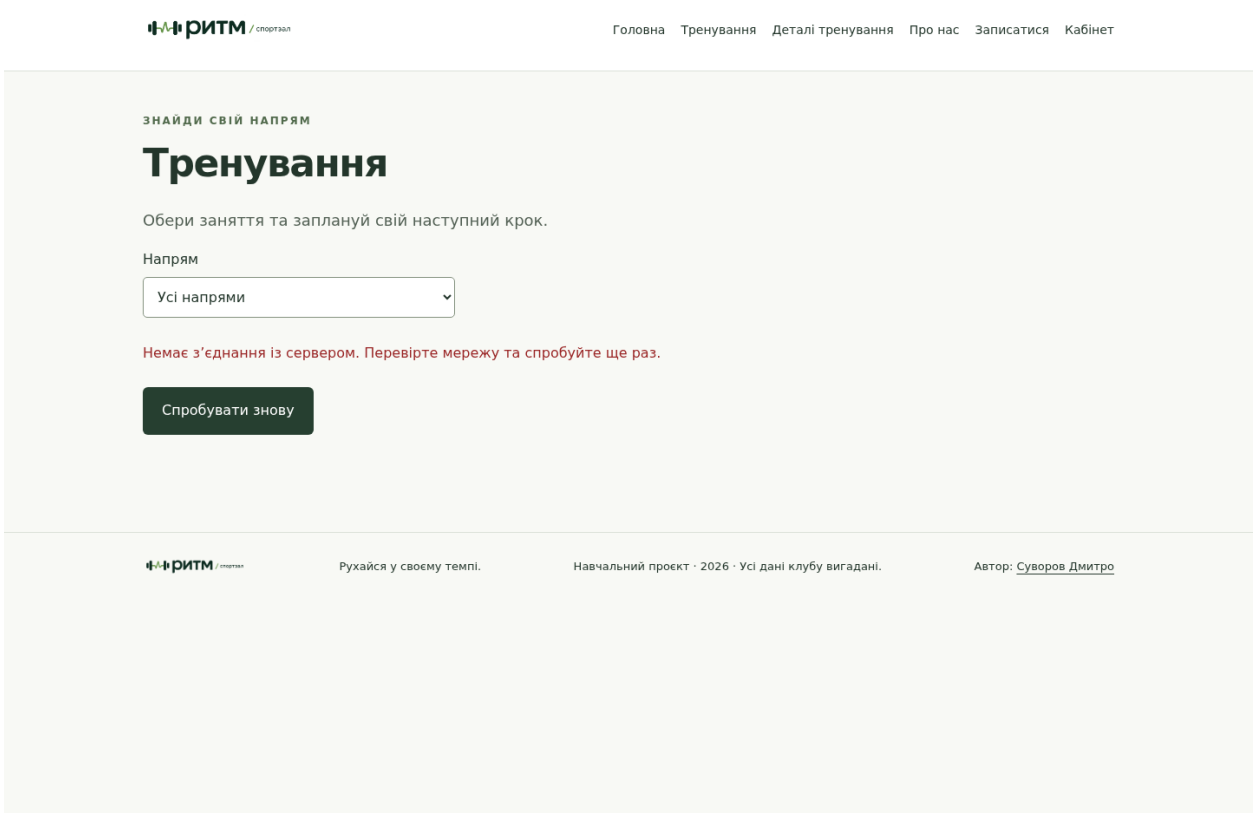
Кабінет із власними записами



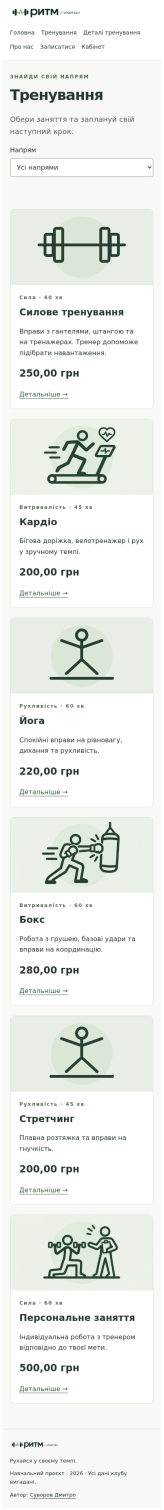
Керування каталогом



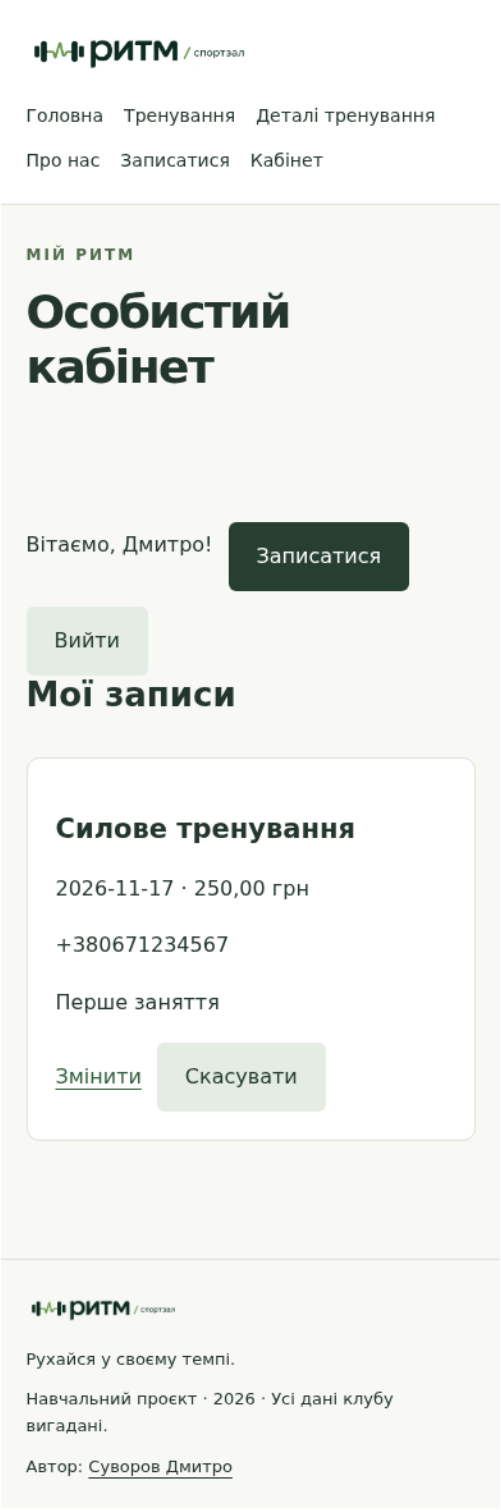
Помилка мережі та повторний запит



Мобільний інтерфейс

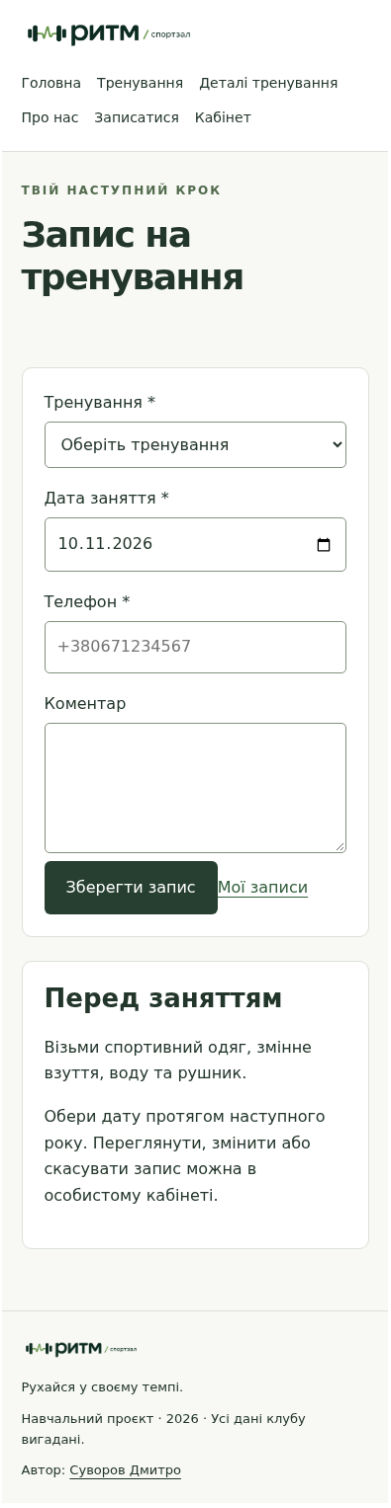


Каталог

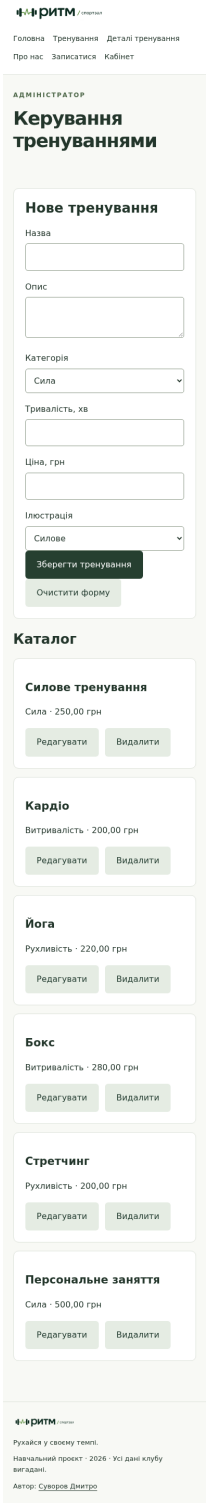


Кабінет

Мобільний інтерфейс



Форма запису



Адміністратор

8. Висновки

Статичний сайт КП №1 розширено до клієнт-серверного застосунку. Реалізовано чотири пов'язані таблиці у 3НФ, REST CRUD двох основних сутностей, реєстрацію та вхід із bcrypt/JWT, перевірку власника й ролей. Клієнт працює з даними через Fetch, показує стани завантаження та помилок. API перевірено через Insomnia, інтерфейс — у двох браузерах. Підготовлено SQL/DBML, документацію API та відтворювану інструкцію запуску. Матеріали КП №1 збережено окремо для демонстрації розвитку проєкту.

9. Посилання та матеріали

- Репозиторій: https://github.com/Dima018/Gym_Website.
- Дошка задач: https://github.com/users/Dima018/projects/13.
- Документація Express: https://expressjs.com/en/guide/error-handling.html.
- Параметризовані запити pg: https://node-postgres.com/features/queries.
- ER-модель для імпорту: `database/schema.dbml`.
- Інструкція ручної перевірки: `docs/API.md`.